Subscription Management System: Architecture and SRS

Brightloop Software | Odoo Hackathon | Version 1.0 | 31 October 2026 | Prepared by Swarnalakshmi (solo participant)

1. Introduction

1.1 Purpose

This document defines the architecture and the software requirements for the Subscription Management System, a single platform that runs products, plans, quotations, subscriptions, recurring invoices, discounts, payments and revenue reporting for Brightloop Software. It is the baseline for development.

1.2 Scope

In scope: authentication and roles, product and variant catalogue, plans, taxes, quotation templates, subscription lifecycle, automatic invoicing, discounts, payment recording, customer portal, and a finance dashboard.

Out of scope: online payment gateways (payments are recorded manually), multi-currency, and accounting-ledger integration.

1.3 Definitions

Term	Meaning
Subscription	A customer's recurring purchase of products under a plan
Plan	Price, billing period, minimum quantity and availability dates
Billing period	One cycle of a plan: daily, weekly, monthly or yearly
Billing run	The process that creates invoices for all due subscriptions
Outstanding	Invoice total minus payments received
Overdue	A Confirmed invoice with outstanding amount greater than 0 and a due date in the past

1.4 References

Subscription_Management_System.pdf (problem statement) and Odoo_Hackathon_Participant_Document.pdf (submission rules).

2. Overall Description

2.1 Users and roles

Capability	Admin	Staff	Customer
Create staff accounts	Yes	No	No

Capability	Admin	Staff	Customer
Manage products, variants, plans, taxes	Yes	View only	No
Manage discounts	Yes	No	No
Manage quotation templates	Yes	View and use	No
Create quotations and subscriptions	Yes	Yes	No
Generate, confirm, cancel, send, print invoices	Yes	Yes	View and print own
Record payments	Yes	Yes	No
Finance dashboard and reports	Yes	Yes	No
Customer records	Yes	Yes	Own profile
Subscriptions and invoices	All	All	Own only

The problem statement fixes two rules: only the admin adds staff, and customers see only their own subscriptions and invoices. The remaining staff boundaries are design decisions.

2.2 Technology stack

Layer	Choice
Frontend	React with Vite, Material UI, React Router, Axios, Recharts
Backend	Node.js, Express
Database	MongoDB Atlas with Mongoose (replica set, so transactions are available)
Auth	JWT and bcryptjs
Scheduling	node-cron plus an admin 'Run billing now' action
Email	Nodemailer (password reset and invoice sending)
Hosting	Vercel (frontend), Render (backend), Atlas (database)

2.3 Assumptions and constraints

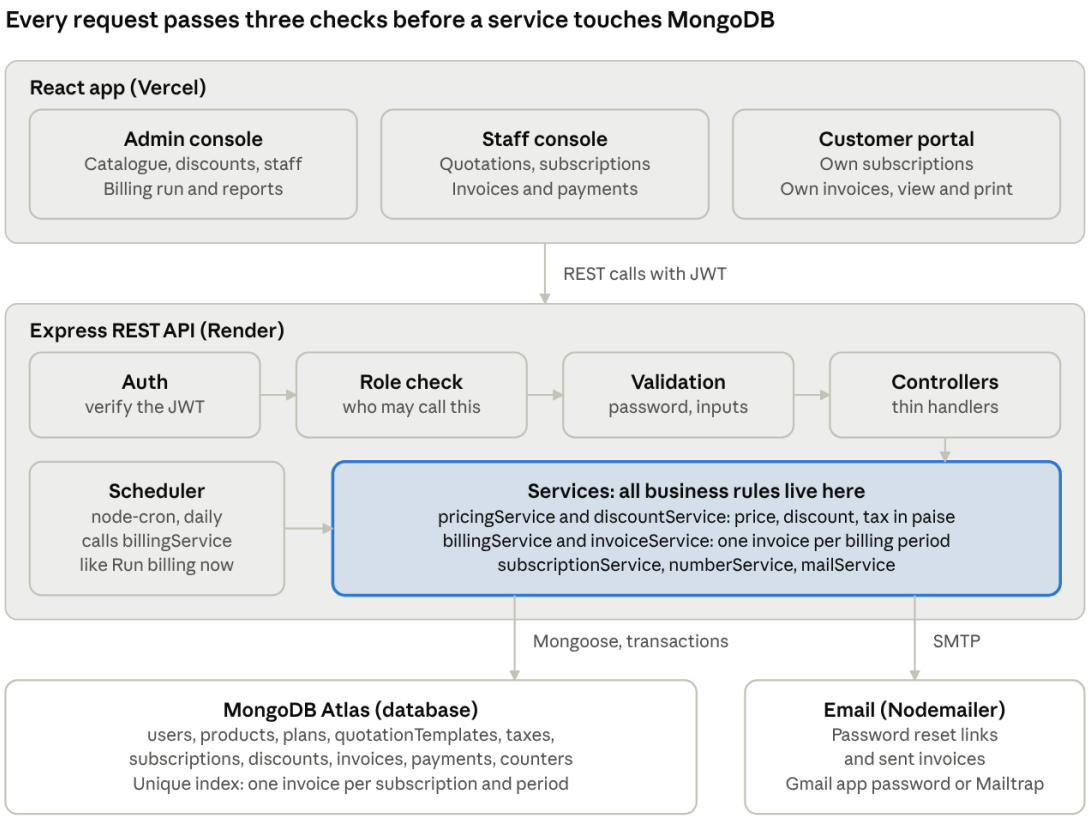
- Currency is INR. All amounts are stored as integers in paise to avoid floating-point errors.
- Taxes are added on top of prices (tax-exclusive).
- Subscriptions are billed in advance, at the start of each billing period.
- Only one GitHub branch (main) is used, as the hackathon requires.
- The MERN stack must be confirmed as permitted with the mentor before development (see section 13).

3. System Architecture

3.1 Layers

1. React single-page application with three experiences: Admin, Staff and Customer Portal, all behind protected routes.
2. Express REST API: routes, then middleware (auth, role, validation), then controllers, then services (business logic), then Mongoose models.
3. MongoDB Atlas storing all data.
4. Scheduler: a daily node-cron job calls the same billing service as the admin 'Run billing now' button.

Request path: Browser, Axios with JWT, Express route, authMiddleware (verify token), roleMiddleware (check role), validator, controller, service, model, MongoDB, JSON response.



system architecture · 3 layers, 8 components

Requests move down the layers: each passes auth, role and validation checks before a service touches MongoDB. The daily scheduler and the admin's Run billing now button both start the same billing service.

3.2 Backend structure

```
backend/
  config/db.js
```

```
models/       User, Product, Plan, QuotationTemplate, Subscription,
              Tax, Discount, Invoice, Payment, Counter
controllers/  auth, user, product, plan, template, subscription,
              tax, discount, invoice, payment, report
routes/       one file per controller
middleware/   auth.js, role.js, validate.js, error.js
services/     billingService, invoiceService, pricingService,
              discountService, subscriptionService, numberService, mailService
jobs/         billingJob.js
utils/        money.js, dates.js
server.js
```

3.3 Frontend structure

```
frontend/src/
  components/   Layout, Sidebar, DataTable, ProtectedRoute, StatusChip
  context/      AuthContext
  services/     api.js (Axios instance with token interceptor)
  pages/        auth, dashboard, products, plans, templates,
                subscriptions, invoices, discounts, taxes, payments,
                customers, staff, portal
  routes/       AppRoutes.jsx
```

3.4 Deployment

Frontend on Vercel, API on Render, database on MongoDB Atlas. Secrets live in environment variables and are never committed. Render's free tier sleeps when idle, so the billing run is idempotent and catches up on missed periods, and the admin can trigger it manually during the demo.

4. Database Design

All dates are MongoDB Date values. Every collection has createdAt and updatedAt.

Collection	Key fields
users	name, email (unique, lowercase), passwordHash, role (admin, staff, customer), phone, company, address, isActive, resetTokenHash, resetTokenExpires
products	name, type, salesPrice, costPrice, notes, taxId, isActive, variants[{attribute, value, extraPrice}]
plans	name, productId, price, billingPeriod (daily, weekly, monthly, yearly), minQuantity, startDate, endDate, autoClose, allowEarlyClose, allowPause, allowRenew, isActive
quotationTemplates	name, validityDays, planId, lines[{productId, variant, quantity}], isActive
subscriptions	number, customerId, planId, status, startDate, expiryDate, paymentTerms (immediate, net15, net30), lines[{productId, variant, quantity, unitPrice, taxId, taxAmount, amount}], discountIds[], quotationValidUntil, nextBillingDate, isPaused, templateId

Collection	Key fields
taxes	name, ratePercent, isActive
discounts	name, type (fixed, percentage), value, minPurchase, minQuantity, startDate, endDate, usageLimit, usageCount, appliesTo (products, subscriptions), productIds[], subscriptionIds[], isActive
invoices	number, subscriptionId, customerId, billingPeriodKey, periodStart, periodEnd, issueDate, dueDate, status (draft, confirmed, paid, cancelled), lines[{productId, description, quantity, unitPrice, discountAmount, taxAmount, amount}], subtotal, discountTotal, taxTotal, total, amountPaid, sentAt, appliedDiscountIds[]
payments	invoiceId, customerId, method (cash, bank transfer, card, UPI, cheque), amount, paymentDate, note, recordedBy
counters	key, seq (atomic number sequences for SUB-, INV- and QUO-numbers)

Required indexes

Collection	Index	Purpose
users	email unique	One account per email
invoices	subscriptionId + billingPeriodKey unique	Nothing billed twice
invoices	customerId + status, status + dueDate	Portal, overdue and outstanding queries
subscriptions	status + nextBillingDate	Billing run
subscriptions	customerId + status	Portal and filters
payments	invoiceId, paymentDate	Payments per invoice, revenue by date
discounts	isActive + startDate + endDate	Eligibility lookup

5. Workflows

5.1 Subscription lifecycle

Required statuses: Draft, Quotation, Confirmed, Active, Closed. Pausing is not a status; it is a flag on an Active subscription, so the five required statuses stay exact.

From	To	Trigger	Condition
(new)	Draft	Staff creates subscription, optionally from a template	Customer and plan selected
Draft	Quotation	Staff sends quotation	Has at least one line; sets quotationValidUntil (template

From	To	Trigger	Condition
			validityDays, default 30)
Quotation	Confirmed	Staff confirms for customer	Today is not after quotationValidUntil
Confirmed	Active	Activation on or after start date	Sets nextBillingDate to start date
Active	Closed	Expiry date reached	Plan autoClose is true
Active	Closed	Admin or staff closes early	Plan allowEarlyClose is true
Draft, Quotation	Closed	Cancelled or quotation expired	None

Pause and resume: allowed only on Active subscriptions whose plan has allowPause. While paused, no invoices are created and nextBillingDate does not move. Renew: allowed only if plan allowRenew; extends expiryDate by the original term and keeps the subscription Active.

5.2 Invoice lifecycle

Draft, then Confirmed, then Paid. An invoice can also be Cancelled from Draft or Confirmed, but only while it has no payments. Overdue is derived, not stored.

From	To	Trigger
(new)	Draft	Created by billing run or manually from a subscription
Draft	Confirmed	Staff confirms; invoice becomes payable and dueDate is fixed
Confirmed	Paid	Payments total equals invoice total (automatic)
Draft, Confirmed	Cancelled	Staff cancels; discount usage is released

Sending (email) and printing (print-friendly page, optional PDF) are available on Confirmed and Paid invoices.

5.3 Billing run (no double billing, no skipping)

1. Select subscriptions with status Active, isPaused false and nextBillingDate on or before today.
2. For each, loop while nextBillingDate is on or before today and not after expiryDate, so missed periods are caught up.
3. Build the invoice: lines from the subscription, price calculation (5.4), customer details, dueDate from payment terms.
4. Inside one MongoDB transaction: insert the invoice with billingPeriodKey (subscriptionId plus period start date), increment discount usage guarded by usageLimit, and advance nextBillingDate by one billing period.
5. If the unique index rejects the insert, the period was already billed; skip it safely and continue.
6. If nextBillingDate passes expiryDate and the plan autoClose is true, set status Closed.
7. Return a run summary: created, skipped, failed (with reasons).

5.4 Price calculation (backend only)

1. Line unit price: for the plan's own product, plan.price; for add-on products, product.salesPrice. Add the selected variant's extraPrice (for example Odoo brand adds 560).
2. Line subtotal is quantity times unit price. Quantity must be at least plan.minQuantity.
3. Discounts: each line gets at most one product discount (the largest saving) and the invoice gets at most one subscription discount (the largest saving). A subscription discount is spread across lines in proportion to line subtotal, with any rounding remainder on the last line.
4. Tax per line is the discounted line amount times the tax rate, rounded to the nearest paise.
5. Line amount is discounted subtotal plus tax. Invoice totals are sums of lines. The frontend never supplies totals.

A discount is eligible when it is active, today is between startDate and endDate, usageCount is below usageLimit (if set), it targets the product or subscription, the subtotal meets minPurchase, and the quantity meets minQuantity.

5.5 Payments

Staff records a payment against a Confirmed invoice with method, amount, date and note. Amount must be greater than 0 and not exceed the outstanding balance. The invoice amountPaid is updated in the same transaction; when amountPaid equals total the invoice becomes Paid. Partial payments are therefore supported.

5.6 Authentication and password reset

Registration creates a customer account only. Login returns a JWT. Forgot password always returns the same message regardless of whether the email exists; if it does, a random single-use token (stored hashed, valid for 1 hour) is emailed as a reset link. Reset applies the password rules, stores a new hash and invalidates the token.

5.7 End-to-end demo flow

Admin creates tax, product with variant, and plan. Admin creates a discount and a quotation template. Staff creates a quotation from the template, confirms it, and activates the subscription. Billing run creates the invoice with tax and discount. Staff confirms and sends it. Staff records payment; invoice becomes Paid. The customer logs in and sees only their own subscription and invoice. The finance dashboard shows the updated revenue.

6. Functional Requirements

6.1 Authentication and users

- FR-01: A visitor can register with name, email and password; each email maps to one account (case-insensitive).
- FR-02: Password must be more than 8 characters and contain an uppercase letter, a lowercase letter and a special character, enforced on frontend and backend.
- FR-03: Users can log in and out; the API issues a JWT with an expiry.
- FR-04: Forgot password sends a reset link to the registered email; the link expires after 1 hour and works once.
- FR-05: Only the admin can create staff accounts.
- FR-06: Role-based access applies on every API route and every frontend route.

- FR-07: Admin can list, deactivate and reactivate staff and customers.

6.2 Products, variants, taxes

- FR-08: Admin can create, edit and deactivate products with name, type, sales price, cost price and notes.
- FR-09: A product can have variants (for example brand) each with an extra price added to the base price.
- FR-10: Admin can create and manage tax rules (name, percentage, active) and assign a tax to a product.
- FR-11: Staff can view products and taxes but not change them.

6.3 Plans

- FR-12: Admin can create plans with price, billing period (daily, weekly, monthly, yearly), minimum quantity, and start and end availability dates.
- FR-13: Per plan, admin sets whether it auto-closes at its end, can be closed early, can be paused, and can be renewed.
- FR-14: Only plans within their availability dates can be selected for new subscriptions.

6.4 Quotation templates and subscriptions

- FR-15: Admin can create quotation templates with a plan, product lines and a validity in days.
- FR-16: Staff can create a subscription from scratch or from a template, which prefills plan and lines.
- FR-17: Each subscription gets a unique number (SUB-00001 format), with customer, plan, start date, expiry date and payment terms.
- FR-18: Lines show product, variant, quantity, unit price, tax and amount, calculated by the backend.
- FR-19: Subscriptions move through Draft, Quotation, Confirmed, Active and Closed per section 5.1, and invalid transitions are rejected.
- FR-20: A quotation cannot be confirmed after its validity date.
- FR-21: Pause, resume, early close and renew are available only where the plan allows them.
- FR-22: Every user can see the current status of a subscription they are permitted to view.

6.5 Invoicing and tax

- FR-23: Invoices are created automatically from subscriptions with customer details, lines, taxes and totals filled in.
- FR-24: A scheduled daily billing run and an admin 'Run billing now' action both invoke the billing service.
- FR-25: Each subscription is billed exactly once per billing period; missed periods are caught up.
- FR-26: Staff can confirm, cancel, send (email) and print any invoice, following the lifecycle in 5.2.
- FR-27: Invoice numbers are unique and sequential (INV-00001 format).
- FR-28: Taxes are calculated automatically from admin-defined rules.
- FR-29: Staff can create a manual invoice for a subscription if needed, still subject to the one-per-period rule.

6.6 Discounts

- FR-30: Only the admin can create, edit and deactivate discounts.
- FR-31: A discount is fixed or percentage, with minimum purchase and/or minimum quantity, start and end dates, and a usage limit.
- FR-32: A discount can target specific products or whole subscriptions.
- FR-33: Eligible discounts are applied to the next invoice automatically and show as discount lines.
- FR-34: A discount stops applying when it expires or its usage limit is reached.

6.7 Payments

- FR-35: Staff can record a payment with method, amount, date and note against an invoice.
- FR-36: Payment amount cannot exceed the outstanding balance.
- FR-37: The invoice becomes Paid automatically when fully settled.
- FR-38: Staff can list invoices by Paid, Outstanding and Overdue.

6.8 Customer portal

- FR-39: A customer sees only their own subscriptions, invoices and payments.
- FR-40: A customer can view and print their invoices and see subscription status and next billing date.
- FR-41: A customer can update their own profile and cannot access any other customer's data by changing an ID.

6.9 Dashboard and reports

- FR-42: The dashboard shows active subscription count, revenue collected, outstanding amount and overdue invoices.
- FR-43: Filters by date range, status, customer, plan and payment method.
- FR-44: A summary view and a monthly revenue chart.
- FR-45: Overdue invoices are listed with customer, due date, days overdue and amount.
- FR-46: Search by subscription number, invoice number and customer name.

7. Business Rules

- BR-01: One account per email address.
- BR-02: Only the admin creates staff; self-registration always creates a customer.
- BR-03: Customers can access only records where customerId equals their own id; this is enforced in the query, not just the UI.
- BR-04: Subscription statuses are exactly Draft, Quotation, Confirmed, Active, Closed.
- BR-05: Pause, early close and renewal depend on each plan's flags.
- BR-06: Closed subscriptions are never billed or reopened.
- BR-07: At most one invoice per subscription per billing period, enforced by a unique index.
- BR-08: Billing period length follows the plan: 1 day, 7 days, 1 calendar month or 1 calendar year; month-end dates use the last day of shorter months.
- BR-09: Invoice dueDate equals issue date plus payment terms (immediate, 15 or 30 days).
- BR-10: Only Draft or Confirmed invoices without payments can be cancelled; Paid invoices are final.
- BR-11: A discount cannot make a line or invoice negative.

- BR-12: Discount usageCount increases once per invoice that uses it and is released if that invoice is cancelled.
- BR-13: Quantity must be at least the plan's minimum quantity.
- BR-14: All money is stored in integer paise and computed on the server.
- BR-15: A tax or product that is deactivated does not alter existing invoices.
- BR-16: Issued invoices keep a snapshot of prices, taxes and discounts at the time of billing.

8. API Specification

All routes are under /api. 'A' is admin, 'S' is staff, 'C' is customer. List endpoints accept page, limit, search and filters.

Method	Endpoint	Roles	Purpose
POST	/auth/register	Public	Customer registration
POST	/auth/login	Public	Login
POST	/auth/forgot-password	Public	Send reset link
POST	/auth/reset-password	Public	Set new password
GET	/auth/me	All	Current user
POST	/users/staff	A	Create staff
GET	/users	A, S	List users and customers
PATCH	/users/:id/active	A	Activate or deactivate
GET, POST	/products	S read, A write	List, create product
PUT, DELETE	/products/:id	A	Update, deactivate product
GET, POST	/taxes	S read, A write	List, create tax
PUT	/taxes/:id	A	Update tax
GET, POST	/plans	S read, A write	List, create plan
PUT	/plans/:id	A	Update plan
GET, POST	/templates	S read, A write	Quotation templates
PUT	/templates/:id	A	Update template
GET, POST	/subscriptions	A, S, C own	List, create subscription
GET, PUT	/subscriptions/:id	A, S, C own read	Detail, edit while Draft
POST	/subscriptions/:id/send-quotation	A, S	Draft to Quotation
POST	/subscriptions/:id/confirm	A, S	Quotation to Confirmed

Method	Endpoint	Roles	Purpose
POST	/subscriptions/:id/activate	A, S	Confirmed to Active
POST	/subscriptions/:id/pause, /resume	A, S	Pause flag
POST	/subscriptions/:id/renew	A, S	Renew
POST	/subscriptions/:id/close	A, S	Close
GET, POST	/discounts	A	List, create discount
PUT	/discounts/:id	A	Update, deactivate
GET	/invoices	A, S, C own	List invoices with filters
GET	/invoices/:id	A, S, C own	Invoice detail and print view
POST	/invoices/generate	A, S	Manual invoice for a subscription
POST	/billing/run	A	Run billing now
POST	/invoices/:id/confirm, /cancel, /send	A, S	Invoice actions
GET, POST	/payments	A, S	List, record payment
GET	/reports/summary	A, S	Dashboard totals
GET	/reports/revenue	A, S	Monthly revenue series
GET	/reports/overdue	A, S	Overdue invoices

9. Non-Functional Requirements

- NFR-01 Performance: every page and API response completes in under 2 seconds with thousands of subscriptions.
- NFR-02 Pagination: every list endpoint is paginated (default 20, maximum 100).
- NFR-03 Indexing: the indexes in section 4 exist and filters use them; reads use lean queries.
- NFR-04 Reports: dashboard totals use MongoDB aggregation, not in-memory loops.
- NFR-05 Security: passwords hashed with bcrypt, JWT secret in environment variables, helmet and rate limiting on auth routes, CORS restricted to the frontend origin.
- NFR-06 Validation: all inputs validated on the server; errors return consistent JSON.
- NFR-07 Integrity: multi-document changes (billing, payment) use transactions.
- NFR-08 Idempotency: re-running the billing run never duplicates invoices.
- NFR-09 Privacy: .env, secrets and customer data are excluded from GitHub.
- NFR-10 Usability: responsive layout; status shown with coloured chips; clear error messages.
- NFR-11 Auditability: records store createdBy, updatedBy and key status-change timestamps.
- NFR-12 Maintainability: controllers stay thin and business logic lives in services.

- NFR-13 Time handling: dates stored in UTC; billing compares calendar dates in a single configured timezone (Asia/Kolkata).
- NFR-14 Availability of demo: seed script creates an admin, sample staff, customers, products, plans and discounts.
- NFR-15 Testing: unit tests for price calculation, discount eligibility and billing-period logic; manual test of each role.

10. Screens

Area	Screens
Public	Login, Register, Forgot password, Reset password
Admin	Dashboard, Staff, Customers, Products, Plans, Templates, Taxes, Discounts, Subscriptions, Invoices, Payments, Reports
Staff	Dashboard, Customers, Subscriptions, Invoices, Payments, Reports; read-only catalogue
Customer	My subscriptions, My invoices (view and print), My profile

11. Build Plan and Git Checkpoints

Push to main after each stable milestone, roughly hourly, with messages such as feat: add authentication. Never commit broken code just to meet the clock.

Step	Milestone	Commit
1	Repo setup, folders, .gitignore, SRS in docs/, Express and React skeletons, Atlas connection	chore: initialize project
2	User model, register, login, JWT, role middleware, password rules, seed admin	feat: authentication and roles
3	Forgot and reset password with email; staff creation	feat: password reset and staff
4	Taxes, products with variants, plans	feat: catalogue and plans
5	Quotation templates, subscriptions, lifecycle	feat: subscriptions and quotations
6	Pricing service, discounts	feat: pricing and discounts
7	Invoices, billing run, cron, unique-period protection	feat: automatic invoicing
8	Payments, outstanding and overdue	feat: payments
9	Dashboard, reports, filters, charts	feat: reports dashboard
10	Customer portal and access tests	feat: customer portal
11	Performance check, seed 5,000 subscriptions, indexes, bug fixes	perf: indexes and pagination

Step	Milestone	Commit
12	Deploy, final README, demo video (5 to 6 minutes, open access)	docs: final submission

12. Problem Statement Traceability

Scene in the problem statement	Requirements
A new customer signs up	FR-01 to FR-07, BR-01 to BR-03
Launching a new plan	FR-08 to FR-14, BR-05, BR-08
Turning a quote into a subscription	FR-15 to FR-22, BR-04, BR-06
Billing day	FR-23 to FR-29, BR-07, BR-09, BR-10
A customer asks for a better price	FR-30 to FR-34, BR-11, BR-12
The money comes in	FR-35 to FR-38
The finance lead, at the end of the month	FR-42 to FR-46, NFR-01 to NFR-04
Customer portal	FR-39 to FR-41, BR-03

13. Items to Confirm

- MERN permitted: the participant guide does not name a required stack. Confirm with your mentor (Samit) or the Helpdesk before building.
- Mentor GitHub ID: add the one shown on the portal (sbbh-odoo, and hackathon-odoo if it is the evaluator account) as repository collaborators.
- Problem statement selection on the portal is permanent once confirmed.
- Staff permission boundaries in 2.1 and the one-discount-per-line rule in 5.4 are design decisions; change them only if your mentor says otherwise.
- Email provider for password reset and invoice sending (Gmail app password or Mailtrap) must be ready by Step 3.

14. Approval

Role	Name	Date
Participant	Swarnalakshmi	31-10-2026
Mentor	Bhadiyadra Samit Bharatbhai	-